

Group Project Report – PART B Keyword Driven Testing

Andrew Truong, Joey Gonzalez, Hubert Tran, Toan Tran, Vance Tran, Gerrit

Vanderstoel, Courtney Vu

CPSC 463-01 - Software Testing

Professor Ning Chen

Department of Computer Science

California State University, Fullerton

Fall 2025

1.0 Introduction

Now that we're familiar with how to test odoo from the backend, we must now test it from the frontend to make sure there aren't any errors happening from the user's side. Since we don't have access to the codebase, we must use a different program and testing method that can be implemented without access. This is why we had to learn how to conduct keyword-driven testing since it works on the odoo user interface itself.

Extending beyond backend validation is a critical step to ensure any software system functions correctly as a complete product. Backend testing does not fully represent how users interact with the whole application. End users engage with software through graphical interfaces, and any issues in the front end can impact the software's usability, even if the backend operates flawlessly. Frontend testing focuses on validating user interactions, visual elements, and system responses from the perspective of someone using the application in a browser. This includes verifying that pages load correctly, actions trigger the expected behavior, and workflows function smoothly from start to finish.

Keyword-driven testing provides a structured and flexible approach to frontend testing. By abstracting test actions into readable keywords, it allows testers to design and execute test cases without needing to write extensive programming logic for each scenario. Using keyword-driven testing, we can build automated testing frameworks to evaluate real-world applications. For this project, we implemented Selenium keyword-based testing on the ERP system of Odoo 15, specifically for the hospital module.

2.0 Setup Struggles

One of the biggest issues we encountered involved ChromeDriver compatibility, as the ChromeDriver version must match the Chrome browser version exactly. Otherwise, cryptic errors emerge that can be difficult to diagnose. Unlike typical software dependencies that allow some version flexibility, browser drivers demand precise version alignment. Thus requiring developers to monitor and update their testing infrastructure constantly.

Beyond version matching, the Robot Framework setup demands careful attention to system configuration and permissions. ChromeDriver must be placed in the correct system directory with proper executable permissions set. The team had to work with specific Python versions (3.7-3.9) due to Ubuntu 22.04 LTS compatibility issues, ensuring these Robot Framework packages installed in the right virtual environment also adds another layer of complexity. A package installed in the wrong Python environment simply won't be found when tests attempt to run. The nature of web UI testing also introduces unique synchronization and stability challenges. Unlike the unit tests in Part A that interact directly with Python code and database transactions, Robot Framework tests must navigate a dynamic web interface where elements load asynchronously and page structures can shift. Finding the correct XPath selectors for web elements requires inspecting Odoo's HTML structure through browser developer tools, but these paths can become brittle if the UI changes even slightly. The presentation emphasizes that the Odoo server must be running before executing tests—a seemingly obvious requirement, but easy to overlook and frustrating to debug when tests fail with connection errors rather than clear "server not found" messages. These timing and environmental dependencies make UI testing inherently more fragile than the isolated unit tests, requiring more careful orchestration of the testing environment.

Topic 3: Keyword-Driven Test Case Design

The keyword-driven test case design used for this project focuses on modeling real user interactions with the Odoo 15 hospital module. For our testing logic we used custom keywords that represent meaningful user actions such as logging in, navigating modules, creating records, and validating results. We also design our test cases to capture documentation to be used in the future. We designed these test cases to resemble user workflows while being reusable and maintainable.

The implementation began with creating a comprehensive test file called `test_om_hospital.robot`, which is structured to separate configuration settings, reusable keywords, and our test cases. The configuration section contains test variables and settings, including the application title and environment-specific parameters. These settings can be easily modified without altering the test logic itself. This setup also ensures consistency across all test cases, because every test will rely on the same configuration parameters in this file.

User interactions were implemented as reusable custom keywords. One of the primary keywords we implemented was 'Login To Odoo', which handles opening the browser, validating the page title, entering credentials, and submitting the login form. This keyword is reused across all test cases that require authentication, ensuring that login behavior is consistent and reducing redundant code. Any changes to the login process can be handled within this keyword without impacting each test case.

Test cases were designed to validate both functional workflows and input validation in the hospital module. The positive test case of creating a new patient verifies that a patient record can be successfully created by navigating to the hospital module, clicking the create button,

entering valid patient details, and saving the record. We included screenshots that capture the before and after form submission to provide visual confirmation of each step and to be used as documentation later on. We also had negative testing to evaluate how the system handles invalid input. In the negative test case, a patient record is submitted with a negative age value. The test case checks whether an error message or validation popup appears and verifies whether the system incorrectly allows the record to be saved. We designed test cases that can not only be used as confirmation of correct behavior, but also to detect functional bugs in frontend validation logic.

Conclusion

This project successfully demonstrated the implementation of keyword-driven testing using Robot Framework and Selenium to validate the frontend functionality of Odoo 15's hospital module. While we encountered significant technical challenges related to ChromeDriver compatibility, Python environment configuration and the inherent complexity of UI testing, these obstacles provided valuable learning experiences in managing real-world testing infrastructure.

The keyword-driven approach proved to be an effective methodology for frontend testing, enabling the creation of maintainable and reusable test cases that accurately model user workflows. By abstracting test actions into readable keywords, the framework enables efficient test case design without requiring extensive programming for each scenario. The implementation of both positive and negative test cases—including patient record creation and input validation—demonstrated the framework's capability to verify correct system behavior while detecting potential bugs in frontend validation logic.

Ultimately, this project reinforced the critical importance of comprehensive frontend testing as a complement to backend validation. Only through testing the complete user experience can software teams ensure that applications function correctly from the end user's perspective, providing confidence in both system functionality and usability. While the difficulties in compatibility added a bit of difficulty to the testing process, the end result remained a success.

References

Robot Framework User Guide. (n.d.). Robotframework.org.

<https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>

Sasiwara. (2021, November 19). *Using XPath in Robot Framework*. Medium.

<https://medium.com/@sasiwara.boon/using-xpath-in-robot-framework-61511408df4c>

with. (2014, February 24). *Keyword-driven testing with Squish and Robot Framework*.

Knowledgebase.qt.io.

<https://qatools.knowledgebase.qt.io/squish/integrations/robot/keyword-driven-testing-squish-robot-framework/>